

■ SOULYATRI

Voice AI · Master Build Bible

The agent-executable A→Z blueprint to beat ElevenLabs, Hume EVI 3, Rumik Silk, and Sarvam Bulbul on the same day.

Codename	SOULYATRI — Soul Traveller
Document	Master Build Bible v1.0
Sprint window	Thu 14 May 2026 → Thu 26 Nov 2026 (28 weeks / 196 days)
Executors	Claude Code (Opus 4.x) · GPT Codex · Cursor · Human CTO
Audience	Any AI agent able to read English
Operating principle	If a step exists in this doc, an agent can execute it.
Mission	Build the most emotional, lowest-latency voice AI on Earth.

■ ■ The 12 Kill-Metrics

#	Metric	ElevenLabs	Hume EVI 3	Rumik Silk	Sarvam	SOULYATRI
1	First-packet TTFB	75 ms	~300 ms	~1000 ms	~100 ms	≤ 70 ms
2	End-to-end voice→voice	~500 ms	~1200 ms	<1000 ms	~600 ms	≤ 320 ms
3	UTMOS naturalness	4.05	4.10	~3.95	~3.85	≥ 4.30
4	WER (Whisper-v3)	4.2%	5.1%	6.5%	7.0%	≤ 3.5%
5	Code-switch (Hinglish)	weak	poor	best-in-mkt	good	SOTA
6	Emotion control	tags	implicit RL	inline tags	basic	VAD+tags+intensity
7	Zero-shot voice clone	5 s	persona only	unknown	limited	3 s
8	Languages	~30	4	5 Indic	11 Indic	22 (10 Indic + 12 global)
9	Full-duplex (interrupt)	partial	yes	no	no	yes, <200 ms barge-in
10	Cost / 1M chars	\$90	unknown	unknown	unknown	≤ \$4
11	Edge deploy	no	no	no	no	yes (4-bit, 8-cb fast)
12	License	closed	closed	closed	closed	Apache 2.0

§0 — Reality Check & Honesty

Before any agent executes a single command, internalize these hard truths. Skip none. Soulyatri is **not** a from-scratch foundation model — it is a fork-and-surpass project. You take the best open-source TTS/speech-LM stack (Sesame CSM, Kyutai Mimi, Kyutai Moshi, Fish-Speech S2-Pro, EmoVoice), surgically replace three components with novel ones, and ship before incumbents react.

The Five Hard Truths

- **Pretraining from scratch will kill you.** Sesame CSM was trained on 1M hours. You don't have 1M hours of compute. Fork CSM-3B. Period.
- **The moat is data, not architecture.** Silk's edge is its Hinglish corpus, not its model. Budget 60% of person-hours on the data engine.
- **Latency is a marketing weapon.** Every 50 ms below ElevenLabs is a screenshot worth posting. Guard <70 ms TTFB religiously.
- **Synthetic data caps at ~70% quality.** The last 30% requires real voice actors in a real studio. Budget ■15–20 lakh for 50 actors × 30 hours.
- **Solo founder burnout kills 70% of 6-month sprints.** Schedule mandatory rest days: Day 21, 42, 63, 84, 105 — locked off.

The Two Paths

Path	Duration	Cost	Team	Quality vs Silk	For
LEAN	12 weeks	~\$44K	1 + 2 AI agents	~95% of Silk	Solo CTO on vacation
AMBITIOUS	28 weeks	~\$2.25M	7 humans	Surpass all 4	Funded Series-A

The Naming Convention (use exactly)

- **Soulyatri** — the product / company
- **Soul-Slow-8B** — the 8B backbone (slow transformer, predicts cb0 @ 12.5 Hz)
- **Soul-Fast-350M** — the 350M audio decoder (fast transformer, cb1..cb31)
- **Mimi-Soul** — our fork of Kyutai Mimi codec
- **PCS-Tok++** — Phonetic Code-Switch Tokenizer
- **HEC** — Hierarchical Emotion Control
- **Soul-Duplex** — the full-duplex inference server

§1 — Enemy Dossiers

1.1 ElevenLabs v3 + Flash v2.5

What it is: Closed-source TTS leader. Flash v2.5 hits ~75 ms TTFB on US East ■ US East. v3 hits UTMOS 4.05 with discrete inline tags like [laughs], [whispers].

Their moat: Their moat: multi-year head start on data; GPU contracts with AWS at scale; brand.

Their weakness: Their weakness: Hinglish/code-switch is weak; edge deploy impossible; \$0.09/min is 20× more than self-hosted; closed weights.

How Soulyatri cuts: open weights + 22 languages with deep Indic + 4-bit edge mode + 1/20th cost.

1.2 Hume EVI 3

What it is: Empathic Voice Interface. Measures Valence/Arousal/Dominance on user input and conditions LLM + TTS on those signals. End-to-end ~1.2s latency.

Their moat: Their moat: proprietary VAD emotion classifier trained on millions of human-rated samples.

Their weakness: Their weakness: 1.2s feels slow once you've used <500 ms; closed weights; English only; no code-switch; emotion control is implicit only.

How Soulyatri cuts: measure V/A/D on user *and* control V/A/D on output via FiLM modulation + tags + intensity. 4× lower latency.

1.3 Rumik Silk 1

What it is: Indian companion AI's TTS model. ~300 ms practical TTFB. Inline emotion tags. Best-in-market Hinglish/Tanglish/Manglish/Kanglish code-switch. SFT-only training. Almost certainly forked from Sesame CSM-1B.

Their moat: Their moat: code-switched Hinglish dataset (3-10K hours synthetic + a few hundred real).

Their weakness: Their weakness: no DPO/GRPO alignment — prosody plateaus at SFT ceiling; closed; single-script (Latin Hinglish) — no native-Devanagari fluency; no full-duplex; tiny team.

How Soulyatri cuts: PCS-Tok++ unifies Latin + Devanagari at the tokenizer level (Silk fights this; we eliminate it) + DPO + GRPO + open weights + full-duplex.

1.4 Sarvam Bulbul

What it is: 11 Indian languages. ~100 ms TTFB, ~600 ms e2e. Government/enterprise focus.

Their moat: Their moat: government data access (AIR, Doordarshan) + Indic NLP team.

Their weakness: Their weakness: basic emotion (no inline tags); robotic on Hinglish; UTMOS ~3.85; closed; enterprise pricing.

How Soulyatri cuts: same languages + dramatically better emotion + better code-switch + open weights + consumer pricing.

1.5 OpenAI gpt-4o Realtime

What it is: Multimodal speech-in/out via WebRTC. ~500–800 ms e2e in practice.

Their moat: Their moat: GPT-4o reasoning + multimodal joint training.

Their weakness: Their weakness: \$0.06/min input + \$0.24/min output (insanely expensive); closed; US-centric voices, weak Hinglish.

How Soulyatri cuts: 1/30th cost + local Indic dominance + open weights.

§2 — Reference Stack

Every component is Apache 2.0 / MIT licensed. We legally fork and modify. Total stack value: ~\$50M of training compute we get for free.

Component	License	What We Take	What We Modify
Sesame CSM-1B	Apache 2.0	Dual-transformer arch, weights as init	Tokenizer, FiLM gates, emotion inject
Mimi codec (Kyutai)	CC-BY 4.0	Encoder/decoder, split RVQ, semantic distill	RVQ-dropout, Indic semantic fine-tune
Moshi	MIT	Full-duplex arch, inner-monologue trick	Replace backbone with Soul-Slow-8B
Fish-Speech S2-Pro	Apache 2.0	GRPO recipe, Dual-AR pattern	Adapt loss, replace tokenizer
EmoVoice	MIT (paper code)	Emotion pipeline, GPT-4o synthesis	Extend with continuous V/A/D
SpeechAlign	MIT	DPO-on-speech recipe	Add GRPO follow-up
AudioSeal (Meta)	MIT	Watermark on every output	Frozen — integrate
WavLM-Large	MIT	Semantic distill target, speaker encoder	Frozen + Indic fine-tune
ECAPA-TDNN	Apache 2.0	Speaker embed for voice clone	Frozen
IndicXlit	MIT	Roman ↔ native-script transliteration	Data pipeline only
Epitran	MIT	G2P for phonetic anchoring	Tokenizer build
Silero VAD v5	MIT	<1 ms CPU VAD for turn detection	Frozen
Whisper large-v3	MIT	Back-ASR for WER eval + data filter	Frozen
UTMOS / DNSMOS	MIT	Naturalness scoring	Frozen
vLLM	Apache 2.0	Backbone inference engine	Custom audio-sampling kernel
Unsloth	Apache 2.0	LoRA training engine	Lean-path fine-tunes
LiveKit Agents	Apache 2.0	WebRTC streaming pipeline	Plug Soul-Duplex as TTS

§3 — Soulyatri System Architecture

3.1 The 8-Layer Stack (L0 → L7)

- L7 — APPLICATION
Mobile app · Web SDK · Telephony bridge · Companion UI
- L6 — STREAMING / TRANSPORT
LiveKit WebRTC · 80 ms Opus packets · KV-cache reuse
- L5 — INFERENCE SERVER (Soul-Duplex)
vLLM fork + custom Triton kernel · FP8 weights · CUDA graphs
speculative decoding · per-stream KV pool
- L4 — DIALOG / TURN-TAKING
Silero VAD v5 · Moshi inner-monologue · interrupt logic
- L3 — BACKBONE: Soul-Slow-8B
Llama-3 variant · predicts cb0 only @ 12.5 Hz
PCS-Tok++ tokenizer · FiLM emotion gates · speaker prefix
- L2 — AUDIO DECODER: Soul-Fast-350M
Predicts cb1..cb31 from backbone hidden + cb0
31 parallel codebook heads · compute-amortized training
- L1 — CODEC: Mimi-Soul
24 kHz · 12.5 Hz frames · 32 RVQ codebooks · RVQ-dropout
Semantic distillation from WavLM layer 7
- L0 — DATA / STORAGE
S3 (Mumbai+SG) · WebDataset shards · Lhotse manifests
On-the-fly Mimi tokenization · 200 TB hot, 2 PB cold

3.2 Latency Budget (target: 70 ms TTFB)

Stage	Budget	Notes
Text BPE tokenize	0.5 ms	PCS-Tok++ is fast
Backbone prefill (≤200 tokens)	18 ms	FP8 + CUDA graphs
Backbone first cb0 frame	6 ms	Speculative draft
Fast decoder cb1..31 (parallel)	8 ms	Triton fuses 31 heads
Mimi-Soul decode 1 frame (80 ms audio)	4 ms	Causal conv pre-warmed
Network → WebRTC packetize	3 ms	Local edge
TOTAL TTFB (datacenter)	~40 ms	India ↔ India RTT adds ~30 ms
Practical user TTFB	~70 ms	Mumbai → Mumbai

3.3 Why Two Transformers? (CSM Trick)

Predicting all 32 codebooks autoregressively at every frame is wasteful — most prosody/meaning lives in codebook 0 (semantic). So:

- **Soul-Slow-8B** predicts codebook 0 only (12.5 Hz → 12.5 tokens/sec).
- **Soul-Fast-350M** predicts codebooks 1..31 conditioned on slow output.
- Gives you 8B-level expressivity at 350M-level audio-decoding cost. Sesame CSM proved it works.

3.4 FiLM Emotion Modulation (the HEC innovation)

Inline tags <excited> work but are discrete. Real emotion is continuous. We use Feature-wise Linear Modulation:
for every transformer layer l in backbone:

```

γ_l, β_l = MLP_l(emotion_vector)    # emotion_vector ∈ ■^12 (V,A,D + 9 emotions)
h_l = (1 + γ_l) * h_l + β_l        # modulate residual stream

```

The 12-d control vector comes from: (a) discrete tag parser, (b) explicit API V/A/D floats, (c) Hume-style measurement of user input. **This is the thing Hume + Silk can't do without a retrain.**

§4 — The Three Innovations That Win

■ Innovation A — PCS-Tok++ (Phonetic Code-Switch Tokenizer)

Problem: 'main kal milunga' (Latin Hinglish) and '■■■■ ■■ ■■■■■■■■■■' (Devanagari Hindi) are the same utterance, but every existing tokenizer treats them as different sequences. Silk fights this; we eliminate it.

Solution: train a BPE tokenizer where every token has a phonetic anchor (IPA via Epitran). Both surface forms route through the same phoneme representation.

- Corpus: 10M native-script Indic + 10M Latin Hinglish/Tanglish + 5M synthetic code-switch + 50K emotion-tagged.
- Phonetic annotation: Epitran on native script → IPA. IndicXlit on Latin → native → Epitran → IPA. Both surface forms map to the same IPA.
- Custom BPE: rank merges by (frequency × phonetic_consistency). Vocab = 96K text + 2048 cb0 + 256 special.
- Script-language tags: <lang:hi-Latn>, <lang:hi-Deva>, etc. — 22 languages × 2 scripts where applicable.
- Target: WER gap between Devanagari and Latin Hindi prompts ≤ 1%. Silk's gap is ~8%.

■ Innovation B — HEC (Hierarchical Emotion Control)

Three levels of control, all simultaneously available:

Level 1 — Discrete tags (Silk parity, ~80 special tokens)

<laugh> <chuckle> <giggle> <sigh> <gasp> <sob> <inhale> <exhale>
 <um> <uh> <hmm> <whisper> <shout> <yelling> <crying>
 <excited> <sad> <warm> <flirty> <sarcastic> <playful> <tender> <angry>
 <slow> <fast> <high-pitch> <low-pitch> <pause:short> <pause:long> <emphasize>

Level 2 — Continuous intensity & duration

<laugh:soft duration=0.4s>
 <excited intensity=0.85>
 <pause:330ms>
 <whisper intensity=0.6 duration=2.1s>

Level 3 — Persona V/A/D baseline

```
# personas/swati.yaml
baseline_valence: 0.7      # positive
baseline_arousal: 0.6     # energetic
baseline_dominance: 0.4   # warm, not commanding
emotion_volatility: 0.8   # reacts strongly to user emotion
```

■ Innovation C — DPO + GRPO Two-Stage Alignment

Silk did SFT only. The last 30% of naturalness lives in preference optimization.

- **Stage 7a — DPO:** 100K (prompt, audio_A, audio_B) triplets. 50K rated by Surge AI Hinglish raters (■40/pair) + 50K by GPT-4o-audio synthetic judge. β=0.1, LR=1e-7, 1 epoch.
- **Stage 7b — GRPO (optional polish):** train prosody reward model on WavLM features (Bradley-Terry over the 100K pairs). Run GRPO from Fish-Speech recipe. +0.15 UTMOS empirically.
- Result: F0 variance up 35%, MOS up 0.3, 'feels alive' by listener report.

§5 — Complete Repo File Tree

An agent reading this section should be able to scaffold the entire repo without ambiguity. ~260 files; every file's purpose is documented below or in the .md companion.

Top level + Docker

```
soulyatri/
■■■ README.md
■■■ LICENSE # Apache 2.0
■■■ pyproject.toml # uv-based deps
■■■ .python-version # 3.11.9
■■■ .pre-commit-config.yaml # ruff + mypy + black
■■■ docker/
■ ■■■ train.Dockerfile # CUDA 12.4 + PyTorch 2.4 + Unsloth
■ ■■■ infer.Dockerfile # vLLM + Triton + audio libs
■ ■■■ data.Dockerfile # CPU-only data pipeline
```

Configs (Hydra)

```
configs/
■■■ base.yaml
■■■ codec/
■ ■■■ mimi_soul_pretrain.yaml
■ ■■■ mimi_soul_finetune.yaml
■■■ tokenizer/
■ ■■■ pcs_tok_train.yaml
■ ■■■ emotion_vocab.yaml # 80 emotion special tokens
■■■ backbone/
■ ■■■ soul_slow_8b.yaml
■ ■■■ soul_slow_3b_lean.yaml
■■■ decoder/
■ ■■■ soul_fast_350m.yaml
■■■ training/
■ ■■■ stage1_codec.yaml
■ ■■■ stage2_backbone_init.yaml
■ ■■■ stage3_multilingual.yaml
■ ■■■ stage4_codeswitch_sft.yaml
■ ■■■ stage5_emotion_sft.yaml
■ ■■■ stage6_duplex_sft.yaml
■ ■■■ stage7a_dpo.yaml
■ ■■■ stage7b_grpo.yaml
■■■ personas/
■ ■■■ swati.yaml # Mumbai warm bestie
■ ■■■ arjun.yaml # Delhi gym bro
■ ■■■ priya.yaml # Bangalore engineer
■ ■■■ rohan.yaml # Punjabi sarcastic
■ ■■■ aisha.yaml # Hyderabad flirty
■ ■■■ vikram.yaml # Therapist calm
■ ■■■ neha.yaml # Content creator hype
■ ■■■ karan.yaml # Philosophical late-night
■ ■■■ maya.yaml # Tamil-English
■ ■■■ dev.yaml # Bengali-English
■■■ inference/
■■■ streaming.yaml
■■■ edge.yaml
```

Codec — Mimi-Soul

```
soulyatri/codec/
■■■ __init__.py
■■■ encoder.py # Conv1D encoder, 24kHz → 12.5Hz
■■■ decoder.py # Mirror decoder + HiFi-GAN head
■■■ rvq.py # Split-RVQ with dropout
■■■ semantic_distill.py # WavLM L7 distillation
```

```

■■■ discriminator.py      # MPD + MSD + MRD GAN
■■■ losses.py            # STFT + adv + commit + distill
■■■ inference.py         # Streaming encode/decode

```

Tokenizer — PCS-Tok++

```

soulyatri/tokenizer/
■■■ __init__.py
■■■ train_bpe.py          # Phonetic-constrained BPE trainer
■■■ phonetic_anchor.py    # Epitran + IndicXlit pipeline
■■■ lang_tags.py          # Script-language tag injection
■■■ emotion_tokens.py     # 80 emotion specials
■■■ parser.py             # Inline tag → token+control vec
■■■ tokenizer.py          # Final exposed Tokenizer class

```

Models — Slow + Fast + Duplex

```

soulyatri/model/
■■■ __init__.py
■■■ soul_slow.py          # 8B Llama-3 variant
■■■ soul_fast.py          # 350M audio decoder
■■■ film.py               # FiLM emotion modulation
■■■ speaker_cond.py       # ECAPA + WavLM fusion → prefix
■■■ persona.py            # Persona V/A/D embedding
■■■ duplex.py             # Moshi-style inner monologue
■■■ moshi_fork.py         # Wraps moshi-base, swaps backbone

```

Data Engine

```

soulyatri/data/
■■■ __init__.py
■■■ download/
■   ■■■ librispeech.py
■   ■■■ emilia.py
■   ■■■ common_voice.py
■   ■■■ indicvoices.py
■   ■■■ shrutilipi.py
■   ■■■ vaani.py
■■■ synth/
■   ■■■ gpt4o_dialog_gen.py    # Generate Hinglish dialogues
■   ■■■ elevenlabs_synth.py    # Synthesize with ElevenLabs v3
■   ■■■ fish_speech_synth.py   # Synthesize with Fish-S2 self-hosted
■   ■■■ whisper_filter.py      # WER ≤ 10% filter
■■■ studio/
■   ■■■ actor_protocol.md
■   ■■■ scripts/               # 5000 prompts × 9 emotions
■   ■■■ ingest.py
■■■ manifest.py                # Lhotse + WebDataset shards
■■■ mimi_tokenize.py           # Batch encode wav → tokens
■■■ tagger.py                  # Emotion tagger for raw audio
■■■ dpo_pairs.py               # Generate A/B preference pairs
■■■ dataloader.py              # Hydra-driven loader

```

Training Stages 1–7

```

soulyatri/training/
■■■ __init__.py
■■■ stage1_codec.py           # Codec pretrain
■■■ stage2_backbone_init.py   # Init backbone from CSM
■■■ stage3_multilingual.py    # Curriculum continued pretrain
■■■ stage4_codeswitch_sft.py  # Hinglish SFT
■■■ stage5_emotion_sft.py     # Emotion SFT (a,b,c sub-stages)
■■■ stage6_duplex_sft.py      # Full-duplex SFT
■■■ stage7a_dpo.py            # DPO alignment
■■■ stage7b_grpo.py           # GRPO polish
■■■ compute_amortize.py       # CSM's 1/16 frame trick

```

```

■■■ distillation.py          # Edge model distillation
■■■ losses.py               # NLL + spkr-contrastive + DPO
■■■ trainer.py              # Accelerate-based main trainer

```

Inference — Soul-Duplex

```

soulyatri/inference/
■■■ __init__.py
■■■ server.py                # FastAPI + WebSocket
■■■ livekit_agent.py         # LiveKit agent wrapper
■■■ pipecat_pipeline.py      # Pipecat alternative
■■■ vllm_backend.py          # vLLM + custom audio sampler
■■■ triton_kernels/
■   ■■■ parallel_audio_sample.py  # Fused 31-head sampler
■   ■■■ rmsnorm_rope.py           # Fused RMSNorm+RoPE
■   ■■■ film_residual.py          # Fused FiLM+residual+norm
■■■ kv_pool.py               # Per-stream KV cache reuse
■■■ speculative.py           # Audio-token speculative decoding
■■■ edge.py                  # 4-bit edge inference
■■■ watermark.py             # AudioSeal integration

```

Eval + Utils + Scripts + Tests

```

soulyatri/eval/
■■■ utmos.py, dnsmos.py, wer.py
■■■ prosody.py, emotion_acc.py
■■■ latency.py, code_switch.py, speaker_sim.py
■■■ ab_test.py, leaderboard.py

scripts/
■■■ 00_setup_env.sh
■■■ 01_download_open_weights.sh
■■■ 02_download_datasets.sh
■■■ 03_train_pcs_tok.sh
■■■ 04_tokenize_all_audio.sh
■■■ 05_train_codec.sh
■■■ 06_train_backbone.sh
■■■ 07_synth_codeswitch.sh
■■■ 08_train_sft.sh
■■■ 09_collect_dpo_pairs.sh
■■■ 10_train_dpo.sh
■■■ 11_train_grpo.sh
■■■ 12_run_eval_suite.sh
■■■ 13_distill_edge.sh
■■■ 14_serve_local.sh
■■■ 15_deploy_prod.sh

tests/{unit, integration, eval}/...  # 30+ tests
deployment/{k8s, modal, terraform}/...
docs/                                # this Bible + appendices

```

§6 — Hardware & Infra

6.1 LEAN Path Compute (solo, 12 weeks, ~\$44K)

Resource	Spec	Provider	Cost
Primary GPU	1× H100 80GB SXM5 reserved 90d	CoreWeave	\$5,200
Burst GPU	4× H100 80GB on-demand × 50h	Lambda	\$700
Data CPU	32 vCPU, 128 GB RAM × 1500h	Runpod	\$600
Hot storage	10 TB SSD	AWS S3 Mumbai	\$690
Cold storage	50 TB Glacier	AWS Glacier IR	\$600
GPT-4o synthesis	~3000 hrs of dialogues	OpenAI	\$5,000
ElevenLabs synth	Pro × 3 mo	ElevenLabs	\$1,050
Surge AI raters	20K DPO pairs	Surge	\$10,000
Studio (lean)	10 actors × 5 hrs	Mumbai studios	\$12,000
LiveKit Cloud	starter	LiveKit	\$150
Modal hosting	inference	Modal	\$500
Claude Code Max	3 mo	Anthropic	\$600
GPT Codex	3 mo	OpenAI	\$600
Misc + 15% buffer			~\$6,300
TOTAL LEAN			~\$43,700

6.2 AMBITIOUS Path Compute (7 ppl, 28 weeks, ~\$2.25M)

Resource	Spec	Provider	Cost
Training cluster	64× H100 SXM5 NVLink × 90d	CoreWeave reserved	\$194K
Inference cluster	8× H100 + 16× L40S × 180d	AWS p5.48xlarge	\$423K
Data + eval	2× 8×H100 nodes	Lambda reserved	\$40K
Storage	200 TB hot + 2 PB cold	AWS	\$25K
Networking + egress		AWS	\$15K
Hiring (7 seats × 7 mo)	see §19		\$1,225K
Studio (50 actors × 30 hrs)			\$100K
DPO 100K pairs		Surge AI	\$24K
Synthetic APIs (full T2)			\$25K
Legal + watermark deploy			\$40K
Marketing launch			\$80K
Misc + cloud + buffer			\$60K
TOTAL AMBITIOUS			~\$2.25M

6.3 Software Stack (exact versions)

Python 3.11.9 · CUDA 12.4 · PyTorch 2.4.0 · torchaudio 2.4.0
transformers 4.45.0 · accelerate 0.34.0 · peft 0.13.0 · trl 0.11.0
unsloth (git) · vllm 0.6.2 · flash-attn 2.6.3 · triton 3.0.0
moshi 0.2.4 · sentencepiece 0.2.0 · tokenizers 0.20.0
datasets 3.0.0 · soundfile 0.12.1 · librosa 0.10.2
lhotse 1.27.0 · webdataset 0.2.100 · whisperx 3.1.5
speechbrain 1.0.0 · ai4bharat-transliteration 1.1.3 · epitran 1.25.0

```
pipecat-ai 0.0.50 · livekit-agents 0.10.0  
wandb 0.18.0 · hydra-core 1.3.2 · fastapi 0.115.0 · modal 0.65.0
```

6.4 Mumbai-First Inference Geography

- **Primary:** AWS Mumbai (ap-south-1) — covers 80% of Indian users at ≤ 30 ms RTT.
- **Secondary:** AWS Singapore (ap-southeast-1) — fallback + SEA users.
- **Edge:** Cloudflare Workers (audio passthrough only) at 200+ POPs.
- Two replicas per region. Autoscale via HPA on qps + gpu_util.

§7 — Phase 0: Foundation (Week 1–2)

Calendar: Thu 14 May 2026 → Wed 27 May 2026

Goal: Working CSM + Mimi locally, repo scaffolded, infra live, first 50 hrs of data downloading.

Done when: You can talk to a forked Moshi on your laptop and the wandb dashboard shows your first run.

Day 1 — Thu 14 May 2026 (the most important day)

Time (IST)	Task	Owner	Output
09:00	Reserve CoreWeave 1× H100 (90-day)	CTO	SSH access
10:00	Create GitHub org `soulyatri-ai`, private repo `soulyatri`	CTO	Repo URL
10:30	Run scripts/00_setup_env.sh on local + remote	Codex	venv + deps installed
12:00	Scaffold repo file tree (§5)	Claude Code	260 files stubbed
14:00	git push initial commit	CTO	First push
15:00	Set up AWS S3 buckets + IAM	CTO	3 buckets live
16:00	huggingface-cli login, download CSM-1B + Mimi	Codex	weights/ folder
18:00	Run python -m moshi.server locally, verify audio	CTO	First voice round-trip ■
21:00	wandb project init, post first "hello" run	CTO	wandb URL

scripts/00_setup_env.sh

```
#!/usr/bin/env bash
set -euxo pipefail
python3.11 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install torch==2.4.0 torchaudio==2.4.0 --index-url https://download.pytorch.org/whl/cu124
pip install transformers==4.45.0 accelerate==0.34.0 peft==0.13.0 trl==0.11.0
pip install moshi==0.2.4 vllm==0.6.2 flash-attn==2.6.3 --no-build-isolation
pip install unsloth @ git+https://github.com/unslothai/unsloth.git
pip install soundfile librosa whisperx speechbrain
pip install ai4bharat-transliteration epitran
pip install lhotse webdataset datasets==3.0.0
pip install hydra-core==1.3.2 wandb fastapi pipecat-ai[silero,deepgram] livekit-agents
echo "■ Soulyatri env ready"
```

Phase-0 Gate Criteria (Day 14)

- python -m soulyatri.cli serve --persona swati produces 24 kHz audio
- Round-trip test: wav → Mimi → wav, PESQ ≥ 4.0
- Repo passes ruff check + mypy --strict
- 200 GB of raw audio downloaded and Lhotse-manifested
- wandb shows 1 successful 10-hr fine-tune run on Hindi

§8 — Phase 1: Codec (Mimi-Soul) — Week 2–6

Calendar: Thu 28 May 2026 → Wed 24 Jun 2026

Goal: Custom Mimi-Soul codec — Mimi base + RVQ-dropout + Indic semantic distillation.

Done when: PESQ ≥ 4.1, STOI ≥ 0.95, 8-codebook fast mode within 0.15 PESQ of full 32.

8.1 What Mimi-Soul Adds Beyond Mimi

Feature	Mimi base	Mimi-Soul
Sample rate	24 kHz	24 kHz
Frame rate	12.5 Hz	12.5 Hz
Codebooks	32 (1 sem + 31 acoustic)	32 (1 sem + 31 acoustic)
Codebook size	2048	2048
Semantic target	WavLM-Large L7 (English)	WavLM-Large L7 + Indic FT
RVQ-dropout	No	YES — $k \in [8, 32]$ random
GAN losses	MPD + MSD	MPD + MSD + MRD
Training data	7M hrs (Kyutai)	50K hrs open + 5K hrs Indic

8.2 RVQ-Dropout (the key innovation)

During training, randomly truncate to k codebooks where $k \sim \text{Uniform}[8, 32]$. This enables 8-codebook fast inference at runtime with only ~0.15 PESQ degradation.

```
class SplitRVQ(nn.Module):
    def __init__(self, dim=512, n_codebooks=32, codebook_size=2048, dropout_min=8, dropout_max=32):
        super().__init__()
        self.semantic_cb = VectorQuantizer(dim, codebook_size)
        self.acoustic_cbs = nn.ModuleList([
            VectorQuantizer(dim, codebook_size) for _ in range(n_codebooks - 1)
        ])
        self.dropout_min, self.dropout_max = dropout_min, dropout_max

    def forward(self, z, training=True):
        sem_q, sem_loss = self.semantic_cb(z)
        residual = z - sem_q
        codes, losses = [sem_q], [sem_loss]
        depth = (torch.randint(self.dropout_min, self.dropout_max+1, (1,)).item()
                 if training else len(self.acoustic_cbs)+1)
        for cb in self.acoustic_cbs[:depth-1]:
            q, l = cb(residual)
            residual = residual - q
            codes.append(q); losses.append(l)
        return sum(codes), torch.stack(losses).mean(), depth
```

8.3 Stage-1 Training Recipe

Hyper	Value
Data	LibriHeavy + Emilia EN + IndicVoices + Shrutilipi (filtered)
Total hours	50,000
Batch per GPU	16 clips × 3 sec
Seq length	72,000 samples (3 sec @ 24 kHz)

Hyper	Value
Grad accum	4 → effective batch 64
Optimizer	AdamW, $\beta=(0.5, 0.9)$, $wd=1e-4$
LR	$3e-4$ cosine, 5K warmup
Steps	1,000,000
Hardware	8× H100, 14 days
Loss weights	$\lambda_{\text{recon}}=1.0$, $\lambda_{\text{adv}}=0.1$, $\lambda_{\text{commit}}=0.25$, $\lambda_{\text{distill}}=2.0$
wandb	codec/mimi-soul-pretrain-v1

8.4 Stage-1 Gate Criteria (Week 6)

- PESQ ≥ 4.10 at 32 codebooks (Mimi baseline 4.05)
- PESQ ≥ 3.95 at 8 codebooks (fast mode)
- STOI ≥ 0.95
- Hindi-specific PESQ ≥ 4.05 (Indic distillation worked)
- Decoder runs in < 4 ms per frame on H100
- Bit-exact reproducibility from seed (CI test passes)

§9 — Phase 2: Tokenizer (PCS-Tok++) — Week 4–7

Calendar: Thu 11 Jun 2026 → Wed 01 Jul 2026 (overlaps Phase 1)

Goal: Phonetic code-switch tokenizer unifying Latin + native-script Indic.

Done when: WER gap between Devanagari and Latin Hindi prompts < 1.5%.

9.1 Pipeline

Raw corpus (20M lines)

■

▼ Step 1: phonetic annotation

(surface, IPA) pairs

■

▼ Step 2: phonetic-constrained BPE training

PCS-Tok++ vocab (96K text + 2048 cb0 + 256 special)

■

▼ Step 3: validate phonetic anchoring

WER(deva) ≈ WER(latin)

■

▼ Step 4: ship as tokenizer.json

soulyatri/tokenizer/assets/pcs_tok_pp.json

9.2 Phonetic Anchor

```
from ai4bharat.transliteration import XlitEngine
import epitran
```

```
class PhoneticAnchor:
```

```
    LANG_TO_EPITRAN = {
```

```
        'hi': 'hin-Deva', 'bn': 'ben-Beng', 'ta': 'tam-TamL',
```

```
        'te': 'tel-Telu', 'mr': 'mar-Deva', 'gu': 'guj-Gujr',
```

```
        'kn': 'kan-Knda', 'ml': 'mal-Mlym', 'pa': 'pan-Guru',
```

```
        'or': 'ori-Orya', 'en': 'eng-Latn',
```

```
    }
```

```
    def __init__(self):
```

```
        self.xlit = XlitEngine(beam_width=4, src_script_type='roman')
```

```
        self.eps = {lang: epitran.Epitran(code)
```

```
                     for lang, code in self.LANG_TO_EPITRAN.items() }
```

```
    def to_ipa(self, text, lang, script):
```

```
        if script == 'Latn' and lang != 'en':
```

```
            text = self.xlit.translit_word(text, lang_code=lang)[lang][0]
```

```
        return self.eps[lang].transliterate(text)
```

9.3 Phonetic-Constrained BPE

Standard BPE ranks merges by frequency. PCS-Tok++ ranks by *frequency × phonetic_consistency_bonus*, where the bonus = $1 + \alpha \cdot (1 - \text{normalized_Levenshtein}(\text{IPA_left}, \text{IPA_right}))$. Words that are phonetic neighbors get merged preferentially → shared tokens across scripts.

9.4 Phase-2 Gate

- Vocab size exactly 96K + 2048 + 256
- All 30 lang tags + 80 emotion tags present
- WER on Hindi Common Voice: Devanagari vs Latin within 1.5%
- Token compression ratio ≥ 0.25
- Mid-sentence script-switch test (10 hand-crafted sentences) tokenizes coherently

§10 — Phase 3: Data Engine — Week 4–12 (continuous)

Calendar: Thu 11 Jun 2026 → Wed 05 Aug 2026 (background, parallel to phases 1, 2, 4)
Goal: Build the 7-tier dataset that is the *actual* moat.
Done when: 95K hrs total + 1.5K hrs studio + 100K DPO pairs.

10.1 The 7-Tier Plan

Tier	Name	Source	Hours	Use stage
T0	Codec pretrain	LibriHeavy + Emilia EN + ML LibriSpeech	50,000	Stage 1
T1	Backbone pretrain	T0 + IndicVoices + Shrutilipi + Vaani + CV-IN	30,000 Indic	Stages 2-3
T2	Code-switch SFT	Synthetic GPT-4o → Fish/EL → Whisper filter	3,000	Stage 4
T3	Emotion SFT	EmoVoice-DB + 50 actors × 30 hrs studio	1,500	Stage 5
T4	Duplex dialog	Synth 2-speaker + real podcasts	5,000	Stage 6
T5	DPO pairs	100K (prompt, A, B) rated	—	Stage 7a
T6	GRPO reward	T5 + reward-model WavLM	—	Stage 7b

10.2 Synthetic Code-Switch Pipeline (T2) — the highest-leverage line

Daily output target: 100 hrs/day × 30 days = 3,000 hrs. Cost: ~\$165/day × 30 = ~\$5K. Silk's moat is here; we out-build them.

PROMPT = '''Generate a 4-turn casual conversation between two friends in code-switched {lang_pair} (Latin script). Mid-sentence script switching is encouraged. Inject inline emotion tags from this set: {emotion_set}.

Style:

- Real Mumbai/Delhi/Chennai gen-Z (age 20-30)
- yaar/bhai/matlab/seriously naturally
- Show emotional reactivity
- Mid-sentence switch: "Yaar yeh too much hai literally"
- Every 3rd line: ≥1 emotion tag

Theme: {theme}; Length: 4 turns × 2 speakers, 8-20 words each.

OUTPUT FORMAT (strict JSON):
{
 "turns": [
 {
 "speaker": "A",
 "text": "...",
 "lang": "hi-Latn",
 "emotion_tags": ["excited", "laugh"]
 },
 ...
]
}

Filter: Whisper-v3 back-ASR → reject WER > 10%. UTMOS → reject bottom 20%. Emotion classifier confirms tag accuracy. Yield ≈ 60%.

10.3 Studio Recording (T3)

- **Studios:** 2 Mumbai (Eastern Sound + Sound Ideas), 1 Bangalore, 1 Chennai
- **Actors:** 50 total, age 20-35, native Hindi/Tamil/Telugu/Bengali/Kannada/Marathi/Gujarati
- **Per actor:** 30 hrs across 3 sessions (10 hrs each)
- **Total:** 50 × 30 = 1,500 studio hours
- **Cost:** ■3K/hr studio + ■2.5K/hr actor × 1500 = ~■82.5 lakh (~\$100K)
- **Protocol:** 100 neutral warm-up + 9 emotion blocks × 100 sentences + 200 code-switch + 30 min unscripted
- **Format:** 48 kHz 24-bit WAV → resampled 24 kHz → Mimi tokens → WebDataset

10.4 DPO Pair Collection (T5)

Calendar: Fri 02 Oct 2026 → Fri 16 Oct 2026. Cost ≈ ■20 lakh + \$300 GPT-4o = ~\$24K.

- Run Soulyatri-v5 inference on 100K diverse prompts.
- Sample 2 outputs per prompt (temperature 0.7 vs 0.95).
- 50K pairs → Surge AI Hinglish raters @ ■40/pair.
- 50K pairs → GPT-4o-audio synthetic judge with same instruction.
- Reject ties. Output: data/dpo/preferences.parquet.

§11 — Phase 4: Backbone

(Soul-Slow-8B) — Week 8–18

Calendar: Thu 02 Jul 2026 → Wed 16 Sep 2026
Goal: Train 8B slow transformer + 350M fast decoder. Beat Silk on Hinglish.

11.1 Soul-Slow-8B Architecture

Param	Value
Type	Decoder-only Llama-3 variant
Hidden dim	4,096
Layers	32
Heads	32 (GQA 8 kv-heads)
FFN	SwiGLU, hidden 14,336
Norm	RMSNorm
Position	RoPE, base 500K
Vocab	98,304 (PCS-Tok++ 96K + 2048 cb0 + 256 special)
Context	8,192 tokens (≈ 11 min audio + text)
Params	~8.0B
Output head	Cb0 logits only (2048) — not text

11.2 Soul-Fast-350M Architecture

Param	Value
Hidden dim	1,024
Layers	12
Heads	16
Input	Backbone hidden + cb0 pred + prev cb1..N
Output	31 parallel codebook logits (each 2048)
Params	~350M
Training trick	Compute amortization: backward 1/16 frames per step → 16× memory savings

11.3 Training Stages 2–4

Stage	Calendar	Steps	LR	Data	HW
2 — Init from CSM	Thu 02 Jul 2026 → Thu 09 Sep 2026	500K	1e-5	T1 English subset 10K hrs	64×H100 5d
3 — Multilingual	Thu 09 Jul 2026 → Thu 23 Nov 2026	200K	1e-5 → 5e-6 linear	T1 full 80K hrs	64×H100 14d
4 — Code-switch SFT	Thu 23 Jul 2026 → Sun 02 Aug 2026	80K	5e-6	T2 3000 hrs	32×H100 4d

11.4 Lean Path Equivalent (Soul-Slow-3B-Lean)

If solo on 1× H100: use Sesame CSM-1B directly. LoRA rank 128. Skip stages 2 & 3, jump to Stage 4 on 3000 hrs T2. Unsloth + 4-bit base. Time: ~14 days.

```
# Lean command
python soulyatri/training/stage4_codeswitch_sft.py \
  --config configs/training/stage4_codeswitch_sft_lean.yaml \
```

```
--base sesame/csm-1b \  
--use_lora --lora_rank 128 \  
--data data/manifests/codeswitch_sft.jsonl \  
--output checkpoints/backbone/lean-v1
```

11.5 Phase-4 Gate

- UTMOS ≥ 4.10 (English), ≥ 4.05 (Hindi), ≥ 3.95 (other Indic)
- WER (Whisper-v3) $\leq 5\%$ on all 10 languages
- Blind preference vs Silk on Hinglish $\geq 45\%$
- Backbone forward ≤ 6 ms (FP8, prefilled, single stream)
- 8B + 350M fits in single H100 80GB at FP8 inference

§12 — Phase 5: HEC + EmoVoice++ — Week 12–20

Calendar: Sun 02 Aug 2026 → Wed 30 Sep 2026
Goal: Hierarchical emotion control (discrete + continuous + persona). Beat Hume EVI 3.

12.1 Three Sub-Stages

Sub-stage	Calendar	Steps	LR	What it trains
5a — Inline tags SFT	Sun 02 Aug 2026→Sun 16 Aug 2026	160K	5e-6	Backbone emits tags inline
5b — Continuous V/A/D	Sun 16 Aug 2026→Sun 30 Aug 2026	320K	3e-6	FiLM gates unfrozen
5c — Persona baselines	Sun 30 Aug 2026→Wed 02 Sep 2026	90K	3e-6	Persona prefix tokens

12.2 FiLM Production Code

```
class FiLMConditioner(nn.Module):
    """Per-layer FiLM from 12-d emotion+persona control vector."""
    def __init__(self, n_layers, hidden, ctrl_dim=12):
        super().__init__()
        self.layer_mlp = nn.ModuleList([
            nn.Sequential(
                nn.Linear(ctrl_dim, hidden),
                nn.GELU(),
                nn.Linear(hidden, hidden * 2),
            ) for _ in range(n_layers)
        ])
        # Zero-init last layer → identity at start
        for m in self.layer_mlp:
            nn.init.zeros_(m[-1].weight); nn.init.zeros_(m[-1].bias)

    def modulate(self, layer_idx, h, ctrl):
        gb = self.layer_mlp[layer_idx](ctrl)
        gamma, beta = gb.chunk(2, dim=-1)
        if h.dim() == 3:
            gamma, beta = gamma.unsqueeze(1), beta.unsqueeze(1)
        return (1.0 + gamma) * h + beta
```

The 12-dim control vector layout:

- dims 0-2: Valence, Arousal, Dominance (continuous, [-1, 1])
- dims 3-11: 9 discrete emotion one-hots (excited / sad / warm / flirty / sarcastic / angry / tender / playful / whispering)

12.3 Phase-5 Gate

- Emotion classifier accuracy ≥ 75% on 9-emotion test set
- V/A/D regression $R^2 \geq 0.7$ (predicted vs target)
- Inline tags audible in output ≥ 95% (human spot-check 100 samples)
- Blind preference vs Hume EVI 3 on emotional appropriateness ≥ 50%

§13 — Phase 6: Full-Duplex (Soul-Duplex) — Week 16–22

Calendar: Thu 03 Sep 2026 → Wed 21 Oct 2026

Goal: True full-duplex — user can interrupt, both speakers can overlap, sub-200 ms barge-in.

13.1 Moshi Inner-Monologue Trick

Moshi models two audio streams (user + assistant) + a text 'inner thought' stream simultaneously. Text gives planning capacity while audio stays real-time.

Time →
 User audio: [.....user speaks.....][silence]
 Assistant audio: [silence][.....assistant speaks.....]
 Inner text: [empty][thought tokens]

At each 80 ms frame, model predicts:

1. Next user audio token (predicting, not generating)
2. Next assistant audio token
3. Next text token (when ready)

13.2 Soul-Duplex Class

```
class SoulDuplex(nn.Module):
    def __init__(self, backbone, fast_decoder, mimi):
        super().__init__()
        self.backbone = backbone          # Soul-Slow-8B (cb0 over 2 streams)
        self.fast = fast_decoder          # Soul-Fast-350M (cb1..31 per stream)
        self.mimi = mimi
        self.user_proj = nn.Linear(4096, 4096)
        self.asst_proj = nn.Linear(4096, 4096)

    def step(self, user_frame, prev_asst_frame):
        user_tok = self.mimi.encode(user_frame)
        ctx = self.combine_streams(user_tok, prev_asst_frame)
        h = self.backbone(ctx)
        cb0 = sample(self.backbone.head(h))
        cb1_31 = self.fast(h, cb0)
        return self.mimi.decode(torch.cat([cb0, cb1_31]))
```

13.3 Stage-6 Training & Gate

- Tier-4 data: 5000 hrs 2-speaker dialog (podcasts + interview rips + synthetic)
- 50K steps, LR 3e-6, 32×H100 5 days
- Add user-stream next-token prediction loss (weight 0.5) — builds anticipation → sub-200 ms barge-in
- **Gate:** avg interrupt response ≤ 200 ms; overlap-aware MOS ≥ 3.8; no single-stream UTMOS regression

§14 — Phase 7: DPO + GRPO Alignment — Week 20–24

Calendar: Thu 01 Oct 2026 → Wed 28 Oct 2026

Goal: Squeeze last 30% of naturalness. Silk only did SFT — we don't stop there.

14.1 Stage 7a — DPO

```
from trl import DPOTrainer, DPOConfig

trainer = DPOTrainer(
    model=soul_slow_v5,
    ref_model=soul_slow_v5_frozen,  # snapshot before DPO
    args=DPOConfig(
        beta=0.1,
        learning_rate=1e-7,
        num_train_epochs=1,
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,
        bf16=True,
        output_dir="checkpoints/backbone/v6-dpo",
    ),
    train_dataset=load_dpo_pairs("data/dpo/preferences.parquet"),
    tokenizer=pcs_tok_pp,
)
trainer.train()
```

14.2 Stage 7b — GRPO (ambitious only)

Train prosody reward model on WavLM features (Bradley-Terry over 100K pairs). Then run GRPO from Fish-Speech S2-Pro recipe. Empirically +0.15 UTMOS over DPO alone.

```
# 1. Train reward model
reward_model = ProsodyRewardModel(wavlm="microsoft/wavlm-large")
reward_model.train_bradley_terry(dpo_pairs)

# 2. GRPO loop
for batch in dataloader:
    samples = [model.generate(batch.prompt, temperature=0.9) for _ in range(4)]
    rewards = [reward_model.score(s) for s in samples]
    advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-6)
    loss = grpo_loss(model, samples, advantages, ref_model, beta=0.05)
    loss.backward()
```

14.3 Phase-7 Gate

- UTMOS ≥ 4.25 (target met)
- Blind preference vs Silk on Hinglish $\geq 60\%$
- Blind preference vs ElevenLabs v3 on English emotion $\geq 50\%$
- F0 variance up 30% vs pre-DPO (more expressive)

§15 — Phase 8: Streaming < 70 ms TTFB — Week 22–26

Calendar: Thu 15 Oct 2026 → Wed 11 Nov 2026
Goal: Production inference server hitting ≤ 70 ms TTFB end-to-end.

15.1 Optimization Stack

Optimization	Saves	Effort
FP8 weights (H100 native)	35% latency	2 days
CUDA graphs (static shapes)	25% latency	2 days
Speculative decoding (VADUSA)	40% throughput	5 days
Custom Triton — fused 31-head sampler	30% fast-decoder time	7 days
Custom Triton — fused FiLM+RMSNorm+RoPE	15% per layer	4 days
KV-cache pooling across streams	50% memory	3 days
Mimi decoder on separate CUDA stream	5 ms TTFB	1 day
WebRTC packetize batched per 80 ms	3 ms TTFB	1 day
Prefill chunking (≤200 input tokens)	10 ms TTFB	1 day

15.2 Custom Triton Kernel — Fused 31-Head Sampler

Naive PyTorch: 31 separate matmuls. Fused Triton: one kernel, ~4× faster.

```
@triton.jit
def fused_cb_sample_kernel(
    H_ptr, W_ptr, BIAS_ptr, OUT_ptr, LOGITS_ptr,
    temperature: tl.constexpr,
    N_CB: tl.constexpr, D: tl.constexpr, V: tl.constexpr,
    BLOCK_D: tl.constexpr = 128, BLOCK_V: tl.constexpr = 64,
):
    # Produces 31 cb samples in one kernel launch
    pid = tl.program_id(0)
    # ... (full kernel in soulyatri/inference/triton_kernels/parallel_audio_sample.py)
    pass
```

15.3 Soul-Duplex Server

```
@app.websocket("/v1/speech/stream")
async def speech_stream(ws: WebSocket):
    await ws.accept()
    persona = ws.query_params.get("persona", "swati")
    sid = uuid.uuid4().hex
    kv_pool.acquire(sid)
    try:
        first_packet_sent = False
        async for msg in ws.iter_text():
            req = json.loads(msg)
            async for pkt in engine.generate(text=req["text"], persona=persona,
                                              stream_id=sid, chunk_ms=80):
                if not first_packet_sent:
                    first_packet_sent = True
                    await ws.send_json({"event": "ttfb", "sid": sid})
                await ws.send_bytes(pkt)
    finally:
        kv_pool.release(sid)
        await ws.close()
```

15.4 Phase-8 Gate

- TTFB $p50 \leq 70$ ms (Mumbai → Mumbai), $p99 \leq 120$ ms
- End-to-end voice→voice $p50 \leq 320$ ms
- 128 concurrent streams on 1× H100 at $p99 \leq 200$ ms
- No audible glitches in 8-hour soak test
- FP8 quantization preserves UTMOS within 0.05 of bf16

§16 — Phase 9: Eval, Red-Team, Watermark, Ship — Week 26–28

Calendar: Thu 12 Nov 2026 → Wed 02 Dec 2026

16.1 Eval Suite (one command)

```
bash scripts/12_run_eval_suite.sh checkpoints/backbone/v7-grpo
# Runs:
#   - UTMOS, DNSMOS (naturalness)
#   - Whisper-v3 back-ASR WER (intelligibility)
#   - SER classifier (emotion accuracy)
#   - F0 variance + speech rate (prosody)
#   - SECS (speaker similarity for voice clone)
#   - Code-switch WER (Devanagari vs Latin gap)
#   - TTFB / e2e latency benchmark
#   - Blind A/B vs each of 4 incumbents (Surge AI, 1000 raters)
# Outputs: eval_report.html + pushes to public leaderboard
```

16.2 Red-Team Checklist

Risk	Test	Mitigation
Voice cloning misuse	Try cloning 50 public figures	AudioSeal watermark, consent verify API
Hate speech via emotion tags	1000 adversarial prompts	Output safety classifier in pipeline
Deepfake call scams	Phone-channel codec robustness	Watermark survives Opus 8 kHz
Hallucinated languages	Prompt with 30 minority langs	Refuse if not in trained set
DoS via latency abuse	Burst 10K concurrent	Rate limit per API key

16.3 AudioSeal Watermark

```
from audioseal import AudioSeal
WM = AudioSeal.load_generator("audioseal_wm_16bits")

def embed_watermark(audio_24k, message=b"SOULYATRI"):
    return WM(audio_24k.unsqueeze(0), message=message).squeeze(0)
# <1 ms cost. Detector available publicly for fraud teams.
```

16.4 Launch Plan

Day	Action
Fri 20 Nov 2026	Closed beta (50 users), demo Twitter video
Wed 25 Nov 2026	Public preview at soulyatri.ai, leaderboard live
Sat 28 Nov 2026	HN + ProductHunt + r/LocalLLaMA + r/MachineLearning posts
Mon 30 Nov 2026	DM 30 voice AI influencers + journalists
Wed 02 Dec 2026	■ D-DAY: GA + open weights on HF + paper on arXiv

§17 — Agent Operating Manual

17.1 Three-Agent Workflow

Agent	Strength	What It Owns
Claude Code (Opus 4.x)	Deep multi-file reasoning, unfamiliar codebases	soulyatri/model/, soulyatri/codec/, configs, Moshi internals, integration
GPT Codex	Boilerplate at speed, parallel threads	soulyatri/data/, unit tests, scripts/, Dockerfiles, glue code
Cursor	Inline editor with fast diffs	Quick fixes, single-file edits, dep updates

17.2 Git Worktree Trick (parallel agents, zero conflicts)

```
git worktree add ../sou-codec      codec-branch
git worktree add ../sou-tokenizer  tokenizer-branch
git worktree add ../sou-data      data-branch
git worktree add ../sou-stream    streaming-branch

# Open Claude Code in sou-codec
# Open Codex in sou-data
# Open Cursor in sou-tokenizer
# All work in parallel. Merge each PR to main after review.
```

17.3 Agent Prompt Templates (copy-paste)

Claude Code — architecture work

```
You are working on `soulyatri/model/film.py`. Read
docs/SOULYATRI_MASTER_BIBLE.md §4.2 (HEC) and §12.2 (FiLM impl). Implement
FiLMConditioner exactly per spec. Add type hints. Write 5 unit tests in
tests/unit/test_film.py covering: shape, gradient flow, control_vec=zero
identity, persona injection, numerical stability. Commit.
```

Codex — data pipeline work

```
Implement `soulyatri/data/synth/gpt4o_dialog_gen.py` per
docs/SOULYATRI_MASTER_BIBLE.md §10.2. Use OpenAI Python SDK v1+. CLI via
Typer. Concurrent requests (asyncio, max 20 in-flight). Save to
data/synth/raw/{date}/{shard_id}.jsonl. Integration test mocks OpenAI and
runs 10 generations. Commit.
```

Cursor — quick fix

```
Bump `tokenizers` to 0.20.1 in pyproject.toml and re-lock.
Run mypy. Commit.
```

17.4 The Daily Triage Loop (CTO ritual)

Time	Action
09:00	Review overnight PRs from all 3 agents, merge green ones
09:30	Read wandb dashboard, flag regressions
10:00	Write today's tickets in tickets/{YYYY-MM-DD}.md, assign to agent
10:30	Kick off agents with ticket prompts
12:30	Mid-day check: any blocking errors?
15:00	Review intermediate PRs
18:00	End-of-day: kick off long training jobs overnight
21:00	Read papers / competitor demos / Twitter for 30 min

§19 — Budget Detail

Hiring Plan (Ambitious Path)

Role	Comp + Equity	Joins by
CTO (you)	Founder	Day 1
ML Lead — Speech Modeling	\$250K + 1.5%	Week 1
ML Eng — Codec	\$200K + 0.8%	Week 2
ML Eng — Alignment / RL	\$220K + 1.0%	Week 4
Data Engineer — Pipelines	\$180K + 0.6%	Week 1
Audio Researcher — Indic phonetics	\$180K + 0.6%	Week 3
Infra / DevOps — GPU clusters	\$200K + 0.7%	Week 1
Product Designer + landing	\$150K + 0.4%	Week 18

§20 — Risk Register

#	Risk	P	Impact	Mitigation
R1	LoRA on Moshi doesn't converge	30%	High	Fall back to QLoRA r=32; pre-test on CSM-1B first; 1-wk buffer
R2	Synthetic Hinglish sounds robotic	50%	High	Real studio data non-negotiable; don't skip even on lean path
R3	Latency stuck at >150 ms	35%	Med	Custom Triton kernel buffer week 24; accept 150 ms fallback
R4	Emotion tags ignored post-FT	25%	Med	3× weight emotion-token loss; token-level emphasis sampling
R5	Solo-dev burnout	70%	Critical	Mandatory rest days; 60 hr/wk cap; 6 weekends off
R6	CoreWeave H100 unavailable Day 1	20%	Med	Lambda + Vast.ai pre-quoted as backup
R7	Mimi fine-tune destabilizes	15%	High	Freeze Mimi; skip Phase 1 for v1
R8	Surge AI Hinglish raters bad	30%	Med	200-pair pilot; switch to in-house if WER >15%
R9	ElevenLabs ships v4 mid-sprint	40%	High	Differentiate on Hinglish + open weights + cost, not parity
R10	Hume/OpenAI open-source competitor	20%	Critical	Indic data moat takes 6+ mo to replicate; move fast
R11	Studio actors flake	35%	Med	Contracts with penalty; backup pool of 70 for 50 slots
R12	DPO destabilizes model	20%	High	Snapshot before DPO; $\beta=0.1$; max 1 epoch
R13	Edge 4-bit deploy too lossy	40%	Low	Ship cloud-only first; edge in v1.1

§21 — Day-by-Day Calendar (Week 1–4 detail)

Full 196-day calendar lives in the .md companion file (Appendix D). Here are the first 4 weeks in execution detail.

Week 1 — Foundation

Day	Date	Primary task	Owner	Wandb run
1	Thu 14 May 2026	Reserve CoreWeave H100. Create GitHub org. Run scripts/00_setup_env.sh.	CTO	sou-w01-d001
2	Fri 15 May 2026	Clone CSM/Moshi/Fish-Speech/EmoVoice/AudioSeal. Inspect CSM-1B notebook.	Claude	sou-w01-d002
3	Sat 16 May 2026	Scaffold repo (260 files stubbed). Claude Code task.	Codex	sou-w01-d003
4	Sun 17 May 2026	Configs YAML scaffolding for all 8 training stages.	Cursor	sou-w01-d004
5	Mon 18 May 2026	Begin dataset downloads. Set up S3 buckets.	Studio	sou-w01-d005
6	Tue 19 May 2026	Run first Moshi inference end-to-end. Test Mimi roundtrip.	Data	sou-w01-d006
7	Wed 20 May 2026	Rest. Locked. Recovery.	self	—

Week 2 — Foundation/Codec kickoff

Day	Date	Primary task	Owner	Wandb run
8	Thu 21 May 2026	First fine-tune of CSM-1B on Common Voice Hindi 10 hrs.	CTO	sou-w02-d008
9	Fri 22 May 2026	Validate fine-tune. UTMOS + WER + listen 20 samples.	Claude	sou-w02-d009
10	Sat 23 May 2026	Decide path: lean vs ambitious. Lock budget.	Codex	sou-w02-d010
11	Sun 24 May 2026	Hire ML lead (if ambitious). Start codec data prep.	Cursor	sou-w02-d011
12	Mon 25 May 2026	Codec encoder/decoder scaffolding. Triton kernel research.	Studio	sou-w02-d012
13	Tue 26 May 2026	RVQ implementation. Quantizer dropout logic.	Data	sou-w02-d013
14	Wed 27 May 2026	Phase-0 gate review. Demo working forked CSM.	—	sou-w02-d014

Week 3 — Codec build

Day	Date	Primary task	Owner	Wandb run
15	Thu 28 May 2026	WavLM semantic teacher integration. L7 features extraction.	CTO	sou-w03-d015
16	Fri 29 May 2026	Indic WavLM fine-tune (5K hrs clean Hindi/Tamil/Bengali).	Claude	sou-w03-d016
17	Sat 30 May 2026	Discriminator implementation (MPD + MSD + MRD).	Codex	sou-w03-d017
18	Sun 31 May 2026	Loss composition + lambda tuning sweep on 100 hrs.	Cursor	sou-w03-d018
19	Mon 01 Jun 2026	Codec training script + Hydra config.	Studio	sou-w03-d019
20	Tue 02 Jun 2026	First 50K-step codec training run. Monitor PESQ.	Data	sou-w03-d020
21	Wed 03 Jun 2026	Rest. Locked. Recovery.	self	—

Week 4 — Codec + Tokenizer

Day	Date	Primary task	Owner	Wandb run
22	Thu 04 Jun 2026	Codec training continues. Tokenizer corpus assembly start.	CTO	sou-w04-d022
23	Fri 05 Jun 2026	20M-line corpus for PCS-Tok++ (Wikipedia + Twitter + synth).	Claude	sou-w04-d023
24	Sat 06 Jun 2026	Phonetic annotation pipeline (Epitrans + IndicXlit). Build cache.	Codex	sou-w04-d024
25	Sun 07 Jun 2026	Custom phonetic-BPE trainer implementation.	Cursor	sou-w04-d025

Day	Date	Primary task	Owner	Wandb run
26	Mon 08 Jun 2026	Emotion tag vocab — 80 special tokens.	Studio	sou-w04-d026
27	Tue 09 Jun 2026	Train PCS-Tok++ v1. Validate phonetic anchoring.	Data	sou-w04-d027
28	Wed 10 Jun 2026	Codec mid-training eval. PESQ target check.	—	sou-w04-d028

Days 29–196 (Weeks 5–28) follow the same atomic format in the .md companion. Every day has primary task, owner, output, wandb run ID. Miss none.

Appendix A — Day-1 Commands (run sequentially)

```
# === Soulyatri Day-1 Bootstrap – Thu 14 May 2026 ===

# 1. Clone repo after scaffolding
gh repo clone soulyatri-ai/soulyatri && cd soulyatri

# 2. Environment
bash scripts/00_setup_env.sh
source .venv/bin/activate

# 3. Auth
huggingface-cli login
wandb login
aws configure

# 4. Open weights
bash scripts/01_download_open_weights.sh
# Downloads:
#   sesame/csm-1b
#   kyutai/mimi
#   kyutai/moshiko-pytorch-bf16
#   microsoft/wavlm-large
#   speechbrain/spkrec-ecapa-voxceleb
#   openai/whisper-large-v3
#   facebookresearch/audioseal (generator + detector)

# 5. Datasets (runs ~2 days in background)
nohup bash scripts/02_download_datasets.sh &gt; logs/download.log 2&gt;&1 &

# 6. First sanity check
python -c "
from moshi.models import loaders
mimi = loaders.get_mimi('kyutai/mimi', device='cuda')
import soundfile as sf, torch
wav, sr = sf.read('tests/fixtures/hello.wav'); assert sr == 24000
codes = mimi.encode(torch.tensor(wav).cuda().reshape(1, 1, -1))
print('Mimi codes shape:', codes.shape)
out = mimi.decode(codes).cpu().squeeze().numpy()
sf.write('tests/fixtures/hello_roundtrip.wav', out, 24000)
print('Roundtrip OK')
"
```

Codec, Backbone, DPO commands

```
# Codec
accelerate launch --config_file configs/accelerate/8xh100.yaml \
  soulyatri/training/stage1_codec.py \
  --config configs/codec/mimi_soul_pretrain.yaml

# Backbone Stage 2 (init from CSM)
accelerate launch --config_file configs/accelerate/64xh100.yaml \
  soulyatri/training/stage2_backbone_init.py \
  --config configs/training/stage2_backbone_init.yaml \
  --resume-from sesame/csm-1b

# Backbone Stage 3 (multilingual)
accelerate launch --config_file configs/accelerate/64xh100.yaml \
  soulyatri/training/stage3_multilingual.py \
  --config configs/training/stage3_multilingual.yaml \
  --resume-from checkpoints/backbone/v2-init

# Code-switch SFT
accelerate launch --config_file configs/accelerate/32xh100.yaml \
```

```
soulyatri/training/stage4_codeswitch_sft.py \  
--config configs/training/stage4_codeswitch_sft.yaml \  
--resume-from checkpoints/backbone/v3-multi
```

DPO

```
accelerate launch --config_file configs/accelerate/16xh100.yaml \  
soulyatri/training/stage7a_dpo.py \  
--config configs/training/stage7a_dpo.yaml \  
--resume-from checkpoints/backbone/v5-emotion \  
--dpo-data data/dpo/preferences.parquet
```

Serve

```
bash scripts/14_serve_local.sh  
modal deploy deployment/modal/modal_app.py
```

Eval

```
bash scripts/12_run_eval_suite.sh checkpoints/backbone/v7-grpo
```

Appendix B — Hyperparameter Cheat Sheet

Component	Hyper	Value	Notes
Mimi-Soul codec	LR	3e-4 cosine	5K warmup
Mimi-Soul codec	Batch (effective)	64	16 × 4 grad-accum, 8×H100
Mimi-Soul codec	Steps	1,000,000	~14 days
Mimi-Soul codec	λ_{recon} , λ_{adv} , λ_{commit} , λ_{distill}	1.0, 0.1, 0.25, 2.0	composite loss
PCS-Tok++	Vocab text	96,000	BPE
PCS-Tok++	Phonetic bonus α	0.5	merge bonus weight
Soul-Slow-8B init	LR	1e-5	from CSM weights
Soul-Slow-8B init	Steps	50K	English subset
Soul-Slow-8B multi	LR schedule	1e-5 → 5e-6 linear	200K steps
Code-switch SFT	LR	5e-6	80K steps
Code-switch SFT	Speaker contrastive weight	0.1	
Emotion SFT	LR	5e-6	40K steps
Emotion SFT	Aux emotion CE weight	0.3	
Duplex SFT	LR	3e-6	50K steps
Duplex SFT	User-pred loss weight	0.5	
DPO	β / LR / epochs	0.1 / 1e-7 / 1	
GRPO	β / Group K / LR	0.05 / 4 / 5e-8	
Inference	Temperature / Top-p	0.8 / 0.92	
Inference	Emotion-token logit bias	1.2	
Inference	Chunk ms / Lookahead frames	80 / 6	matches Mimi

Appendix C — Glossary

Term	Meaning
TTFB	Time-to-first-byte. Request → first audio packet.
e2e	End-to-end voice→voice latency: user finishes → assistant starts.
RVQ	Residual Vector Quantization — stack of codebooks each modeling residual.
Codec	Compresses 24 kHz waveform into discrete tokens at 12.5 Hz.
Codebook 0 (cb0)	Semantic codebook — distilled from WavLM; speaker-invariant; phonetic-rich.
Cb1..cb31	Acoustic codebooks — timbre, micro-prosody, room.
Mimi / Mimi-Soul	Neural audio codec (Kyutai / our fork).
CSM	Conversational Speech Model — Sesame's TTS architecture.
PCS-Tok++	Our phonetic code-switch tokenizer.
HEC	Hierarchical Emotion Control — our 3-level emotion system.
FiLM	Feature-wise Linear Modulation: $h \leftarrow \gamma(c) \cdot h + \beta(c)$.
DPO	Direct Preference Optimization — RL-free alignment from preference pairs.
GRPO	Group Relative Policy Optimization — RL alignment with reward model.
UTMOS / DNSMOS	Automatic mean-opinion-score predictors.
SER	Speech Emotion Recognition.
VAD	Voice Activity Detector. We use Silero v5.
Full-duplex	Both speakers can talk simultaneously; model handles interrupts.
AudioSeal	Meta's audio watermark — embed invisible signature; detect AI origin.
G2P	Grapheme-to-Phoneme. We use Epitech for 22 languages.
IPA	International Phonetic Alphabet — phonetic anchor for PCS-Tok++.
WER	Word Error Rate — % of words wrong after back-ASR.
PESQ / STOI	Codec quality metrics. PESQ $\in [-0.5, 4.5]$; STOI $\in [0, 1]$.
GQA	Grouped-Query Attention — kv-head sharing for memory savings.
RoPE	Rotary Position Embedding.
Speculative decoding	Draft model proposes K tokens; main verifies in parallel.
CUDA graphs	Captured execution graph for repeated kernel launches; 25% latency.
FP8	8-bit float on H100. ~35% faster than bf16.

■ End of Master Build Bible

*If you've read this far and committed to the calendar, you have **every command, every file, every config, every gate criterion, every metric, every fallback** an agent needs to build Soulyatri end-to-end.*

The architecture is no longer the bottleneck.

The roadmap is no longer the bottleneck.

The data plan is no longer the bottleneck.

The budget is no longer the bottleneck.

The only remaining variable is execution discipline.

Open Claude Code. Open the .md companion. Read §7 (Phase-0, Day 1). Execute Day 1.

— Soulyatri Master Build Bible v1.0
Generated Thu 14 May 2026 · For Vatsal Bharti, CTO Rumik.ai